UNIVERSITATEA ”ALEXANDRU-IOAN CUZA” DIN IAȘI

FACULTATEA DE INFORMATICĂ



LUCRARE DE LICENȚĂ

Simularea interacțiunilor pradă-prădător prin NEAT

propusă de

Daniel Apostu

Sesiunea: iunie/iulie, 2023

Coordonator științific

Lect. Dr. Croitoru Eugen

UNIVERSITATEA ”ALEXANDRU-IOAN CUZA” DIN IAȘI

FACULTATEA DE INFORMATICĂ

Simularea interacțiunilor pradă-prădător prin NEAT

Daniel Apostu

Sesiunea: iunie/iulie, 2023

Coordonator științific

Lect. Dr. Croitoru Eugen

Cuprins

Introducere **2**

Motivație **5**

1 NEAT **7**

1.1 Rețele Neuronale Artificiale 8

1.2 Problemele în TWEANNs și inovațiile în NEAT 9

1.2.1 Codificare 9

1.2.2 Inițializarea populației 10

1.2.3 Problema convențiilor concurente și numerele de inovație 11

1.2.4 Operatori Genetici 11

1.2.5 Speciație 13

1.2.6 Reproducerea 14

2 rtNEAT **15**

2.1 Înlocuirea agentului cel mai puțin adaptat 15

2.2 Generarea copilului 16

2.3 Parametrii de rulare a algoritmului 17

3 Mediul Virtual **18**

3.1 Vizualizarea principala 18

3.2 Mâncarea 19

3.3 Creaturile 20

3.4 Managementul Algoritmului 25

4 Detalii de implementare **27**

4.1 Genomul în NEAT 27

4.2 Rețelele neuronale din NEAT 29

4.3 Algoritmul NEAT 35

5 Parametrii experimentali **40**

5.1 Main Camera si Walls 40

5.2 FoodManager 40

5.3 Prey si Predator 40

5.4 NeatManager 41

6 Rezultate **44**

Concluzii **45**

Introducere

În ciuda faptului că dinamica populației a atras din punct de vedere istoric interesul multor oameni de știință, suntem încă departe de a înțelege cum și de ce populațiile fluctuează în ecosistemele naturale. Motivul principal este că populațiile sunt sisteme complexe, cu proprietăți emergente care sunt imposibil de determinat prin suma părților lor.[Oro13]
Într-un mediu virtual, am putea izola și controla variabilele din mediu pentru a studia anumiți factori specifici care influențează aceasta dinamica. Specific, vom simula și studia interacțiunile prada-prădător, impactul lor reflectându-se în dinamica populației în următoarele moduri:

- **Reglarea populației** - prădarea reprezinta un mecanism regulator atât pentru populația prăzii cat și a prădătorilor, ce previne prada din a ajunge la densități prea mari, iar densitatea prădătorilor e controlata de cea a prăzii.

- **Dinamica ciclica a populației** - interacțiunile prada-prădător pot rezulta în ciclicitatea populațiilor, unde numărul prăzii și a prădătorilor pot sa prezinte fluctuații în timp în mod regulat. În studierea impactului pe care l-ar avea încălzirea globala asupra dinamicii sistemelor ecologice s-a observat ca pe lângă faptul ca aceste cicluri exista, durata lor se micșorează trecând de la temperaturi calde la cele mai reci[DeL20].

- **Dinamici evolutive** - presiunea impusa de mecanismul de prădare poate duce la adaptări evolutive. În mediul natural prăzile pot dezvolta mecanisme defensive(coarne, camuflaj, viteza) iar prădătorii de-asemenea pot suferi schimbări fizice dar și în strategiile de vânătoare.

În aceasta lucrare vom dezvolta un mediu virtual cu o populație de prada și una de prădători, cu scopul de a analiza transformările suferite în populații după criteriile de mai sus. Mediul va fi dezvoltat în motorul de jocuri Unity, și îl vom descrie în ca-

pitolul 3.El va include 4 pereți poziționați pentru a crea o arie închisa. În cadrul acelei arii vor fi generați în mod constant peleți de energie care vor fi consumați de prada, va fi generata populația de prada și populația de prădători. Evoluția organismelor din fiecare populație se va preta pe dezvoltarea comportamentelor, nu a trăsăturilor fizice. Pentru a direcționa aceasta evoluție, vom folosi un algoritm din clasa algoritmilor evolutivi.

De-a lungul dezvoltării familiei de algoritmi evolutivi, apar anumite etape cheie în care se dezvolta ramuri noi. Enumeram câteva din ramuri în următorul mod:

- **Algoritmii Genetici**(GA) - arie de studiu în care pionier este John Holland prin cartea sa "Adaptation în Natural and Artificial Systems", publicată în 1975[Hol92]. În aceasta carte el introduce utilizarea unei populații de indivizi, a operatorilor genetici precum mutația și crossover, și a mecanismelor de selecție inspirate de selecția naturala.

- **Strategii Evolutive**(ES) - dezvoltarea acestei ramuri poate fi atribuita lui Rechenberg și Hans-Paul Schwefel care sau ocupat de studiul ei în anii '60 și '70. În 1973 Rechenberg publica cartea "Evolutionsstrategie"[Rec73], în care documentează cercetarea sa pe probleme de optimizare continua.

- **NeuroEvolutie** - aceasta clasa de algoritmi cauta prin spațiul de ponderi și structuri o rețea neuronala cu performante bune pentru o anumita sarcina. Practic algoritmii de neuro-evoluție permit găsirea unui comportament care facilitează rezolvarea problemelor de **reinforcement learning** complexe[MM95]. De obicei, în cadrul algoritmilor NE, se alege o tipologie(structura de noduri și conexiuni) pentru rețele ce urmează sa treacă prin procesul de evoluție, iar schimbările au loc doar pe ponderile conexiunilor. **NEAT** este o abordare inedita a neuro-evoluției pe parcursul căreia se modifica atât tipologia rețelei cat și ponderile conexiunilor. Astfel este scutit efortul uman de a crea designul unei rețele. La concepția acestei metode a stat Kenneth O. Stanley, cu articolul s-au din 2002 "Evolving Neural Networks through Augmenting Topologies"[SM02]

În cadrul acestei teze, comportamentele fiecărui individ din populație, adică acțiunile pe care le fac la fiecare pas în timp vor fi dictate de cate o rețea neuronala. Ca algoritmi de neuro-evoluție vom folosi **NEAT** si **rtNEAT**. Acești algoritmi vor fi descriși în capitolele 1 și 2.În cadrul unui experiment în care vom include doar speciile de prada, în

care excludem analiza populației ca număr de indivizi, vom identifica care dintre algoritmi determina o progresie mai rapida în materie de acumulare de energie în medie și cata energie acumulează cel mai bun membru al populației. Un experiment similar a fost descris în lucrarea lui T. Gelles și M. Sanchez[Gel16] în care încearcă sa evolueze comportamentul furnicilor de a plasa feromoni ca un mod de a comunica, și demonstrează ca **rtNEAT** are mai mult succes în a evolua acest comportament decât **NEAT**.
Pentru alte experimente în materie de interacțiune prada-prădător putem face referință la lucrarea "An evolutionary approach for interactive computer games"[YLH04]. În cadrul acesteia autorii proiectează un joc numit "Dead End", în care jucătorul are scopul de a ajunge la ieșirea din partea opusa a ecranului de unde acesta este generat, iar o echipa de 8 "Dogs", ai căror mișcări sunt dictate de cate o rețea neuronala, au scopul de al captura înainte ca acesta sa ajungă la ieșire, deci trebuiau sa dezvolte comportamente de urmărire și de cooperare. Pentru evoluția câinilor au fost folosiți 2 algoritmi de învățare proprii, unul off-line și unul on-line. Ambii aduceau schimbări doar ponderilor conexiunilor din rețea, nu și a topologiei rețelelor. Ei au identificat ca agenții antrenați în modul online prezintă o adaptabilitate mare la schimbarea strategiilor de joc a jucătorilor, deci demonstrează eficacitatea metodei împotriva jucătorilor umani.
Datorita concluziilor acestor 2 lucrări, vom susține ipoteza ca **rtNEAT**, care este un algoritm de învățare online, va determina o evoluție mai rapida a indivizilor, în termeni de energie acumulata. De-asemenea, doar **rtNEAT** ne permite sa facem o analiza a populațiilor pe timp îndelungat, deoarece **NEAT** resetează populațiile la fiecare generație. Dacă identificam ca **NEAT**, care este un algoritm off-line, produce o învățare mai rapida, îl vom folosi pentru a evolua comportamentele indivizilor, iar rețelele antrenate astfel le vom integra în **rtNEAT** pentru a face analiza dinamicii populației. În capitolul 4 vom detalia implementarea mediului, a algoritmilor, a rețelelor, și a senzorilor folosiți. În capitolul 5 vom prezenta experimentele pe care vrem sa le conducem și setările pentru algoritmi în cadrul experimentelor. În final, 6 prezentam rezultatele și analizăm care sunt factorii care le-au influențat.

Motivație

Pe parcursul dezvoltării discipline informaticii, natura a servit ca inspirație pentru oamenii de știința care se ocupau de dezvoltarea noilor algoritmi. Astfel a apărut clasa de **algoritmi evolutivi**, din care se branșează mai departe alte familii de algoritmi: **algoritmii genetici**, **strategii evolutive**, **programarea genetica** și **algoritmii de neuro-evoluție**.

Aceasta familie de algoritmi a ajutat la dezvoltarea multor soluții unde algoritmii clasici determiniști sunt prea costisitori sau ineficienți. Un exemplu ar fi o antena X-band evoluata pentru misiunea NASA din 2006 numita Space Technology 5, al cărei design a fost găsit prin intermediul unui algoritm evolutiv[Hor+06].

Pe parcursul timpului, studiul în natura a interacțiunilor dintre prada și prădător a permis extragerea diverselor informații valoroase despre dinamica ecosistemelor. Datorita avansului rapid al inteligentei artificiale și a algoritmilor evolutivi, avem oportunitatea de a cerceta aceste interacțiuni într-un mediu virtual.

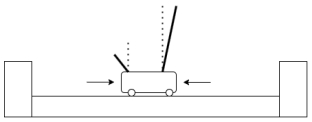
Aceasta teza de licența propune utilizare algoritmilor **NEAT**(NeuroEvolution of Augmenting Topologies) și **rtNEAT**(real time NEAT), plus mediul de dezvoltare de jocuri **Unity**(2D) pentru crearea unei simulări dinamice a interacțiunilor dintre speciile de prada și cele prădător. Înțelegerea acestor interacțiuni este esențiala pentru a descoperi factorii care influențează dinamica populațiilor, coexistența speciilor și stabilitatea ecosistemelor. Utilizând un mediu virtual, putem studia interacțiunile prada-prădător într-un mediu izolat, putem controla variabilele de mediu, și putem repeta experimente de mai multe ori într-un mod consistent(relativ la variabilitatea naturii). Algoritmul NEAT, datorita capacității sale de a evolua atât ponderile cat și topologiile rețelelor neuronale, reprezinta un motor puternic pentru modelarea comportamentelor și a proceselor de luare a deciziilor prădătorilor și prăzii, cu scopul de a supraviețui mai mult. Prin evoluția rețelelor neurale ale prăzii, putem explora comportamentele de căutare a hranei, tactici de evitare și strategii de supraviețuire, iar prădătorii ar tre-

bui sa dezvolte strategii de vânătoare, cu adaptări pentru schimbările din mediu.
Motorul de joc Unity permite sa vizualizam în timp real comportamentele pe care le dezvolta organismele, sa monitorizam noile specii care apar din ambele categorii de organism, sa adaugăm potențial noi obstacole, noi senzori, sau noi abilitați prăzii și prădătorilor, și sa controlam variabilele și viteza simulării.
În concluzie, această teză de licență își propune să îmbine domeniile inteligenței artificiale, al evoluției biologice și al ecologiei, utilizând algoritmul NEAT și Unity pentru a simula interacțiunile dintre pradă și prădători. Prin studierea acestor dinamici ne propunem sa contribuim înțelegerea sistemelor ecologice, cat și la educare populației pe larg asupra acestor interacțiuni prin intermediul vizualizărilor. Ipoteza noastră este ca studiind evoluția populațiilor prăzii și a prădătorilor acestea se vor conforma ecuațiilor Lotka-Voltera.

Capitolul 1

NEAT

Algoritmul NEAT a fost descris pentru prima oara de Keneth O. Stanley și Risto Miikkulainen în articolul lor "Evolving Neural Networks through Augmenting Topologies"[SM02]. În abordările clasice ale **NeuroEvolutiei**(NE), se alege o topologie inițiala a rețelelor neuronale iar în timpul experimentelor algoritmii cauta doar în spațiul ponderilor conexiunilor. Însa topologia(structura) rețelelor are de asemenea un impact asupra comportamentelor pe care le formează o rețea. Un argument bun pentru a aplica evoluția atât asupra ponderilor cat și topologiei vine din partea lui Gruau[Gru93], care susținea ca evoluția structurii unei rețele economisește timpul pe care l-ar fi folosit un om în a decide topologia pentru o anumita problema de NE. A demonstrat asta evoluând topologia și conexiunile unei rețele care a reușit sa rezolve cel mai dificil benchmark de pole balancing de atunci(1996), double pole balancing.



Totuși, aceeași problema a fost rezolvat de 5 ori mai rapid de **Enforced Subpopulations**(ESP)[GM99], astfel, autorii NEAT si-au propus sa demonstreze un argument mai puternic pentru evoluția structurilor, anume ca modificând atât topologia cat și ponderile putem spori cu mult performanta algoritmilor NE. NEAT poate face asta datorita faptului ca minimizează structurile rețelelor pe tot parcursul algoritmului. Au existat pe parcursul timpului mai multe metode de evoluție atât pentru topologiile cat și ponderile rețelelor neuronale, ale căror ideii autorii le-au unit în [SM02] sub termenul de **"TWEANS"**(Topology and Weight Evolving Artificial Neural Networks). Pentru dezvoltarea NEAT Kenneth O. Stanley a adresat un număr de probleme nerezolvat la clasa de algoritmi TWEANS. Înainte de asta ar trebui totuși sa prezentam cum funcționează rețelele neuronale și modul lor de a învața.

1.1 Rețele Neuronale Artificiale

În natura, o rețea neuronala reprezinta o rețea de neuroni interconectați, care sunt responsabili de funcționarea sistemului nervos. Sunt specializați în transmiterea informației prin semnale electrice și au 4 componente principale, dendritele, corpul celular, axonul și butonii terminali.



Fiecare neuron primește informația prin dendrite și le transmite prin axon printr-un semnal electric cu toată puterea sa, dacă a primit destulă informație prin Dendrite. Neuronii sunt conectați intre ei prin sinapse, iar sinapsele își pot adapta numărul de receptori pentru a controla puterea semnalului care vine de la un neuron. O rețea neuronala artificiala este compusa dintr-o serie de noduri conectate intre ele ca un graf. Ca structura, o rețea are un strat de noduri de intrare și unul de noduri de ieșire, și opțional un număr de straturi ascunse în cadrul rețelelor neuronale profunde. Fiecare conexiune are o pondere asociata ei ce reprezinta puterea conexiunii. Aceste ponderi se pot adapta astfel încât rețeaua sa funcționeze conform așteptărilor. Conform Teoriei Aproximării Universale, o rețea neuronala ar putea sa ajungă aproape de a a aproxima orice funcție pentru care este conceputa. Spre deosebire de creier, într-o rețea neuronala pot exista mai multe tipuri de neuroni, determinați de o funcție de activare. Dacă nu folosim o funcție de activare, valoarea unui neuron este reprezentata de o suma ponderata a întrărilor plus opțional un bias al neuronului.



Dacă este folosita o funcție de activare, atunci semnalul nodului va fi outputul funcției de activare, iar input-ul este suma ponderata dintre conexiunile și întrările nodului.

Rețelele neuronale artificiale tradiționale sunt feedforward, adică datele sunt procesate într-o singura direcție, de la input la output, iar învățarea se face prin algoritmul de backpropagation. În documentul care descrie NEAT, nu sunt expuse restricții asu-

pra conexiunilor care se pot forma, deci algoritmul poate duce la generarea rețelelor neuronale recurente.



Acestea pot procesa date secvențiale, deoarece conțin conexiuni ciclice, ce se pot forma intre oricare noduri, inclusiv nodul de la care conexiunea a pornit. La fiecare pas de timp, în RNN nodurile procesează atât intrarea curenta cat și ieșirea nodului recurent anterior, ceea ce permite rețelei sa utilizeze informații din trecut. Pentru antrenare NEAT nu folosește backtracking, ci operatorii genetici.

1.2 Problemele în TWEANNs și inovațiile în NEAT

1.2.1 Codificare

O problema pe care trebuie sa o rezolve orice TWEANN este modul în care specificam genomul unui individ. Aici sunt doua categorii mai mari, **codificare indirecta** și **directa**. Codificarea indirecte de obicei doar specifica reguli pentru crearea fenotipului, și sunt specifice algoritmilor care le folosesc Pentru codificarea directa opțiunile sunt **codificarea binara**, care folosește codificare clasica a grafurilor cu șiruri de biți, precum "Structured Genetic Algorithm"(sGA)[DM92], care reprezinta prin șiruri de biți matricea de conexiuni a rețelei, însa cele mai întâlnite codificări sunt cele de tip graf, care reprezinta grafurile în mod mai explicit. NEAT folosește o codificare directa, din cauza ca codificările indirecte neavând o mapare directa la fenotip, dacă nu suntem foarte familiari cu procesele genetice, pot perturba căutarea în moduri impredictibile și greu de detectat. Codificarea pe care o alegem este foarte importanta, deoarece aceasta afectează costul operației de crossover, astfel ca exista TWEANNs "nonmating", care nici măcar nu folosesc acest operator

Codificarea NEAT a fost proiectata cu scopul de a permite alinierea genom-urilor pen-

tru crossover fără a fi nevoie de o analiza costisitoare a topologiei. Aceasta codificare este reprezentata de o lista de node genes, și o lista de connection genes. Pentru fiecare node gene este specificat id-ul sau și tipul de nod, care clasifica nodurile în noduri de intrare, de ieșire, si noduri ascunse. Primele sunt nodurile de intrare, după care cele de ieșire iar cele hidden sa adaugă la capătul listei în urma mutațiilor. Fiecare conection gene conține: un in-node și un out-node, care exprima nodurile și direcția conexiunii, ponderea asociata cu conexiunea, o valoare boolean care indica dacă conexiunea ar trebui sa fie exprimata în fenotip, și un număr de inovație(prezentam de ce este necesar în secțiunea 1.2.3), care permite găsire genelor care corespund din diferiți indivizi și e una din piesele centrale ale algoritmului.



Figura 1.1: Genotipul și fenotipul exprimat de acesta

1.2.2 Inițializarea populației

La majoritatea algoritmilor din clasa TWEANN, populația inițiala este generata cu topologii aleatorii, pentru a asigura diversitate în structurile rețelelor. Totuși, inițializare în modul asta produce multe probleme pentru TWEANNS:

1. Nu avem nici o garanție ca informația poate ajunge din fiecare intrare la fiecare ieșire, ceea ce poate face ca unele intrări sa fie irelevante. Timp important de procesare va fi pierdut pentru a scăpa de astfel de rețele.

2. Ideal am dori ca soluțiile obținute sa fie minime, adică un număr minim de noduri și conexiuni, deoarece o rețea mai complexa necesita mai mult timp pentru optimizarea ponderilor. Cu inițializarea aleatorie, o mare parte din timp ar fi folosita pentru filtrarea structurilor extra care nu ar fi trebuit sa existe de la început. Evaluarea relelor mai complexe ia mai mult timp, dar prin aceasta inițializare nu avem nici o garanție ca soluția va fi minima, deoarece orice rețea atâta timp cat are un fitness mare poate domina peste celelalte.

Soluția NEAT este sa începapă în mod minim. Populația inițiala are fiecare nod de intrare conectat cu cele de ieșire cu ponderi aleatorii, și nici un nod ascuns, astfel soluția va adăuga structuri noi doar atunci când aceste ajuta la îmbunătățirea fitness-ului. Astfel se asigura minimalitatea nu doar la final dar pe parcursul tuturor generațiilor. Ar trebui sa precizam ca inițial, adăugând unei rețele structuri noi, care nu au fost optimizate, va face ca fitness-ul sa scadă inițial, deci indivizii cu inovații topologice sunt la risc sa dispară, asta fiind un alt motiv pentru care majoritatea algoritmilor TWEANN folosesc inițializarea cu topologii aleatorii. Pentru a evita pierderea inovațiilor, NEAT folsește **"Speciația"**, pe care o vom detalia în secțiunea 1.2.5.

1.2.3 Problema convențiilor concurente și numerele de inovație

Problema convențiilor concurente apare atunci când exista mai mult de un singur mod de a reprezenta o rețea neuronala optimizata pentru o problema. Crossover în cazul a doua astfel de soluții poate duce la pierderea de informație în cadrul noului copil format. Aceasta problema este încă mai grava în cadrul TWEANNs, deoarece soluții pentru o problema pot fi reprezentate prin topologii complet diferite, iar genomul fiecăreia poate avea lungimi diferite, agravând problema convențiilor concurente. Soluția în cadrul NEAT la aceasta problema este inspirata din soluția naturii în cadrul reproducerii sexuale, adică **omologia**, doua gene fiind omoloage dacă ele sunt trăsături ale aceleiași alele. NEAT stabilește omologia dintre gene prin intermediul **marcatorilor istorici**, reprezentați de **numere de inovație**, care permit stabilirea originii istorice a unei gene, iar astfel putem identifica și genele omoloage. Vom prezenta cum apar și cum sunt folosite numerele de inovație în secțiunile 1.2.4 și 1.2.5

1.2.4 Operatori Genetici

NEAT folosește 2 operatori genetici pentru modificarea genomul-ui indivizilor, mutația (fig 1.2) și crossover. Prin mutație, ca și la ceilalți algoritmi de NE, se pot modifica ponderile, prin perturbarea lor sau prin asignarea unei noi valori aleatorii, însa pentru NEAT mutație este și modalitate prin care introducem schimbări structurale. Sunt folosite 2 tipuri de mutație structurala, "add connection", prin care se adaugă o conexiune cu pondere aleatorie intre oricare 2 noduri care nu erau conectate deja, și "add node", care alege o conexiune în mod aleatoriu, și adaugă un nou nod în locul ei. Mai exact, conexiunea aleasa este dezactivata, și se creează o noua conexiune in-

tre nodul de intrare al conexiunii dezactivate și nodul nou creat, cu ponderea de 1, iar conexiunea dintre nodul nou și nodul de ieșire primește ponderea celei dezactivate. Astfel se minimizează efectul inițial al modificărilor structurale, iar ponderile noilor conexiuni vor fi modificate în generațiile viitoare. De fiecare data când produ-



Figura 1.2: Cele 2 tipuri de mutație, ”add connection” și ”add node”

cem o modificare structurala, fiecare gena de conexiune are adăugat și un nou număr de inovație din punct de vedere global. Acest număr ne permite sa stabilim originea istorica a acestei gene. Dacă în cursul unei generații, păstram o lista a inovațiilor care au avut loc iar în genomul a 2 indivizi diferiți va apărea o aceeași gena de conexiune, poate doar cu ponderi diferite, aceste gene vor primi ambele același număr de inovație, pentru ca reprezinta aceeași structura dar care a apărut în mod independent în cadrul fiecăreia. Crossover reprezinta încrucișarea genomului celor 2 părinti, pentru a crea un copil ce are structuri din partea ambilor Acum, datorita marcatorilor istorici, în cadrul crossover doua genomuri pot fi aliniate după genele care au numere de inovație egale, acestea sunt gene ”matching”. Genele care nu sunt matching sunt fie ”disjoint” dacă sunt în limita numerelor de inovație a ambilor părinti, fie ”excess” dacă unul din părinti are numere de inovații mai mari decât cel mai mare număr al celuilalt părinte. În cadrul compunerii copilului, acesta moștenește în mod aleator genele parching ale



Figura 1.3: Procesul de ”matching” pentru doua genomuri diferite

ambilor părinti, însa doar genele disjoint și excess ale părintelui cu cel mai mare ”fit-

ness". În cazul în care ambii părinti au fitness egal, atunci și genele excess și disjoint sunt moștenite aleator. Pentru genele dezactivate, este o probabilitate prestabilita ca o gena sa rămână dezactivata dacă ea e asa în cadrul oricăruia dintre părinti.
Prin intermediul acestor 2 operatori genetici, este introdusa diversitate în cadrul populației. Totuși, de obicei, datorita faptului ca structurile mai mici au mai puține conexiuni, aceste se pot optimiza mai rapid, iar adăugarea de noi structuri, pana acestea sunt optimizate, va scădea din fitness-ul rețelei, deci inevitabil noile inovații ar dispărea rapid dintr-o populație normala, cu tote ca potențial ele ar fi chiar necesare. Din acest motiv în NEAT este nevoie de **Speciație**, pe care o prezentam în secțiunea 1.2.5.

1.2.5 Speciație

Speciația este un mod de a proteja noile inovații topologice apărute. Ideea este de a permite indivizilor sa concureze în propria lor nișă, nu cu întreaga populație. Pentru a detalia, topologiile similare din cadrul populației vor fi încadrate în specii, organismele din aceleași specii concurează în mod principal intre ele, și astfel au timp sa își optimizeze conexiunile. Cum anume sunt determinate speciile și organismele din cadrul lor? Tot cu ajutorul numerelor de inovație. Numărul de gene "desjoint" si "excess" sunt un indicator bun al distantei din punct de vedere al compatibilității intre 2 genomuri. Astfel, autorii NEAT au ajuns la o formula simpla care sa măsoare compatibilitatea ca o distanta intre 2 structuri diferite în NEAT(notam δ) în termeni de număr de gene excess E și număr de gene desjoint D, plus media diferenței dintre genele matching $\overline{W}$:

$$\delta = \frac{c1 * E}{N} + \frac{c2 * D}{N} + c3 * \overline{W} \tag{1.1}$$

Coeficienții $c1$, $c2$ si $c3$ ajustează importanta acestor factori, iar N, care reprezinta numărul de connection genes a genomului cel mai lung, normalizează valorile conform dimensiunii genomului. Prin calculul valorii $delta$ putem desfășura procesul de speciație cu ajutorul unei valori stabilite în prealabil, pragul de compatibilitate δt. Procesul de speciație se desfășoară în următorul mod. Fiecare specie are un genom care o reprezinta. Fie un genom g din generația curenta, acesta este plasat în prima specie cu a cărui reprezentant este compatibil, adică $\delta < deltat$. Dacă nu exista nici o specie compatibila, se formează una noua cu g ca reprezentat al acesteia.

1.2.6 Reproducerea

NEAT folosește un mecanism numit **"explicit fitness sharing"** pentru a evita ca o specie sa devina prea mare și sa ajungă de dimensiunea întregii populații. Practic, toate organismele din aceeași specie împărtășesc fitness-ul speciei lor. Fitness-ul ajustat al organismului i este calculat prin formula:

$$f\prime_i = \frac{f_i}{\sum_{j=1}^{n} sh(\delta(i,j))} \tag{1.2}$$

unde $f\prime_i$ e fitness-ul ajustat al individului i, n e numărul de organisme din populație, iar funcția sh are valoare 0 când $\delta(i,j)$ este peste δ_t, dar 1 altfel. Astfel, partea de jos a fracției reprezinta doar numărul de indivizi din aceeași specie cu organismul i. Fiecărei specii ii este acordat un număr diferit de membri potențiali depinzând de suma a $f\prime_i$ a membrilor speciei După formula:

$$n_k = \frac{\overline{F_k}}{\overline{F_{tot}}}|P| \tag{1.3}$$

unde k reprezinta o specie, n_k e nr de membri asignați speciei, $|P|$ este numărul de indivizi în populație, $\overline{F_k}$ este media fitnessului speciei, adică $\overline{F_k} = \sum_i f\prime_i$, iar $\overline{F_{tot}} = \sum_k \overline{F_k}$ Întreaga populație este apoi înlocuită de copii membrilor mai buni ai fiecărei specii, membrii mai slabi fiind eliminați în prealabil. Scopul final al speciației este protejare inovațiilor topologice, iar sistemul întreg cauta o soluție în cel mai rapid mod posibil, deoarece inovații noi apar și sunt păstrate doar atunci când e necesar.

Capitolul 2

rtNEAT

rtNEAT este o metoda bazata pe NEAT(vedeți capitolul 1), dezvoltată de aceiași autori plus Bobby D. Bryant[SBM05]. Acesta este o adaptare a NEAT, gândită pentru jocuri, pentru evoluția rețelelor neuronale în real-time, permițând agenților controlați de ele sa se îmbunătățească în timpul desfășurării jocului. Pentru a demonstra acest concept de joc, a fost dezvoltat "NERO"(Neuro-Evolving Robotic Operatives)[Uni03], un joc bazat pe rtNEAT, în care un jucător antrenează o echipa de roboti virtuali pentru a se lupta cu robotii altei echipe. Una dintre principalele observații care au permis dezvoltarea rtNEAT este faptul ca într-un joc toată populația joaca în același timp, deci fitness-ul este înregistrat constant, astfel agenții de-asemenea s-ar putea adapta constant. Am considerat acest algoritm potrivit deoarece în mediul nostru virtual, ne dorim ca organismele sa evolueze și sa se adapteze în timp-real, pe baza răspunsurilor pe care le primesc de la diversele interacțiuni cu mediul, astfel am putea vedea o evoluție în termeni de comportamente cat și ca număr de indivizi, deoarece algoritmul NEAT resetează populația la fiecare generație.
În cadrul rtNEAT, la fiecare n intervale de timp, se executa o serie de operațiuni pentru înlocui un organism cu performante scăzute cu copilul dintre 2 organisme mai performante, alese în mod probabilist astfel încât sa prezerveze speciația din NEAT.

2.1 Înlocuirea agentului cel mai puțin adaptat

Prin aceasta operație, se înlocuiește un agent puțin performant cu unul potențial mai performant, dar în asa fel încât se păstreze dinamica de speciație din NEAT. Pentru asta, alegem sa eliminam agentul cu cel mai mic "fitness ajustat", deoarece dacă



Figura 2.1: Ciclul de înlocuiri în cadrul rtNEAT

alegerea sar face după cel mai rău fitness ”neajustat”, ar fi eliminate întâi organismele din specii în care a apărut o inovație topologica noua și care nu a avut timp sa se optimizez, astfel speciația nu ar mai proteja inovația. Fitness-ul ajustat este calculat la fel ca în cadrul NEAT, adică pentru individul i, $f\prime_i = \frac{f_i}{|S|}$, $|S|$ fiind numărul de indivizi din aceeași specie cu i.

Trebuie de asemenea sa ne asiguram ca fiecare individ a supraviețuit destul înainte sa fie eliminat, deoarece altfel ar fi eliminați mereu cei mai noi indivizi, care nu au avut timp sa acumuleze fitness. Din acest motiv, rtNEAT elimina doar agenții care au supraviețuit minimul de timp. Acest parametru e setat experimental. După eliminarea unui agent, este important sa estimam din nou fitnessul mediu $\tilde{F}$, deoarece acesta se schimba, și este folosit în următorul pas al algoritmului.

2.2 Generarea copilului

Din cauza ca nu generam din nou întreaga populație la începutul fiecărei populații, nu poate fi utilizata formula 1.3, dar dinamica speciației poate fi conservata dacă alegem părintele în mod probabilist:

$$Pr(S_k) = \frac{\overline{F_k}}{\overline{F_{tot}}} \tag{2.1}$$

adică probabilitate de a alege o specie anume pentru părinți este proporționala cu raportul dintre fitness-ul mediu al speciei și suma fitness-urilor medii pentru toate speciile. Noul copil este format prin crossover dintre 2 indivizi din specia aleasa plus mutația.

Dacă folosim un prag de compatibilitate dinamic δ_t, atunci nu putem face la fel ca în ”NEAT”, adică sa îl actualizam înainte de crearea următoarei generații, deoarece aproape toată populația va rămâne în aceeași specie. În rtNEAT, este ales un număr de

înlocuiri de indivizi intre care se fac ajustări ale δ_t, de exemplu la fiecare 3 înlocuiri, iar pe baza noului δ_t fiecare individ din populație va fi repartizata din nou la una speciile existente.

2.3 Parametrii de rulare a algoritmului

Toți Pașii algoritmului rtNEAT, pe care i-am discutat mai sus, trebuie executați la intervale regulate, adică la fiecare n ticuri de timp. Parametrul n trebuie ales astfel încât rata de înlocuire sa le permită agenților sa supraviețuiască destul cat sa ajungă la timpul minim m, pentru a putea fi evaluați, adică sa treacă din statutul de neeligibili pentru înlocuire în eligibili, dar și destul de des cat sa nu se încetinească evoluția la un grad frustrant. Pentru a determina parametrii potriviți, autorii au stabilit o lege a eligibilității, care stabilește ce proporție din populație ne putem aștepta sa fie neilegibilă odată ce algoritmul a trecut de primele generații când toată populația este neilegibilă:

$$I = \frac{m}{|P|n} \tag{2.2}$$

unde I e fracția neilegibilă din populație, m e durata minima de viata pe care o poate avea un individ, n este numărul de ticuri intre procesele de înlocuire, iar $|P|$ numărul de indivizi în populație. Practic, cu cat este mai mare populația și numărul de ticuri intre înlocuiri, cu atât este mai greu este înlocuită populația în întregime, deci mai putini agenți neeligibili, iar cu cat m este mai mare, cu atât mai multi vor fi sub pragul eligibilității. Astfel, putem construi o formula pentru n:

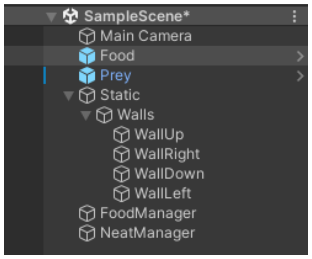
$$n = \frac{m}{|P|I} \tag{2.3}$$

Astfel, parametri pe care trebuie sa ii alegem noi rămân doar m, $|P|$ și I.
Procesele descrise pana acum permit rtNEAT sa replice dinamica NEAT, dar în timp real.

Capitolul 3

Mediul Virtual

În acest capitol vom descrie mediul Unity în care se desfășoară simularea. O scena în Unity este compusa dintr-o mulțime de **GameObjects**, care interacționează intre ele în timpul jocului, sau care prezintă doar aspecte estetice. Mai departe vom prezenta obiectele din cadrul simulării, proprietăților lor, și cum interacționează cu celelalte obiecte din mediu. Structura scenei a fost inspirata din proiectul "Simulated Fish Ecosystem"[Zol21].



3.1 Vizualizarea principala

În mod implicit, înainte sa dam start oricărei simulări, în mediu avem 2 structuri principale: camera(**Main Camera**) și pereții(**Walls**). **Main Camera** stabilește dimensiunea **Game view**-ului (poate fi văzut ca preview în coltul de dreapta jos a imaginii 3.1, unde de-asemenea putem vedea cu o linie gri subțire marginea view-ului), cat și culoarea din background. Pentru view-ul prezentat de camera nu avem o dimensiune standard, deoarece o schimbam depinzând de cantitatea de mâncare și de cate creaturi vrem sa folosim în cadrul unei simulări, însa păstram un aspect de imagine constant de 16:9. Obiectul "Walls" este compus din 4 pereți(reprezentați ca segmente albe în cadrul imaginii 3.1): **WallUp**, **WallRight**, **WallDown** si **WallLeft**. Aceștia sunt obiecte statice, care nu se mișca în timpul simulărilor. Dimensiunea lor de asemenea nu este standard, dar la scalare menținem același aspect de imagine precum camera. Fiecare dintre pereți are atașat un **Box Collider 2D** care este setat ca fiind Trigger, pentru a detecta coliziunile cu alte obiecte, și un **Rigidbody**

2D static plus tag-ul ”Wall”, pentru a putea fi detectați prin intermediul **Raycast**(detalii în secțiunea 3.3). Atunci când scalam pereții pentru diferite experimente o facem astfel încât marginile interioare ale pereților sa fie în afara marginilor view-ului camerei, deoarece la generarea cu coordonate aleatorii ale mâncării și ale creaturilor, acestea sunt generate intre marginile game view-ului, iar pereții fiind în afara lui evitam generarea obiectelor în pereți.

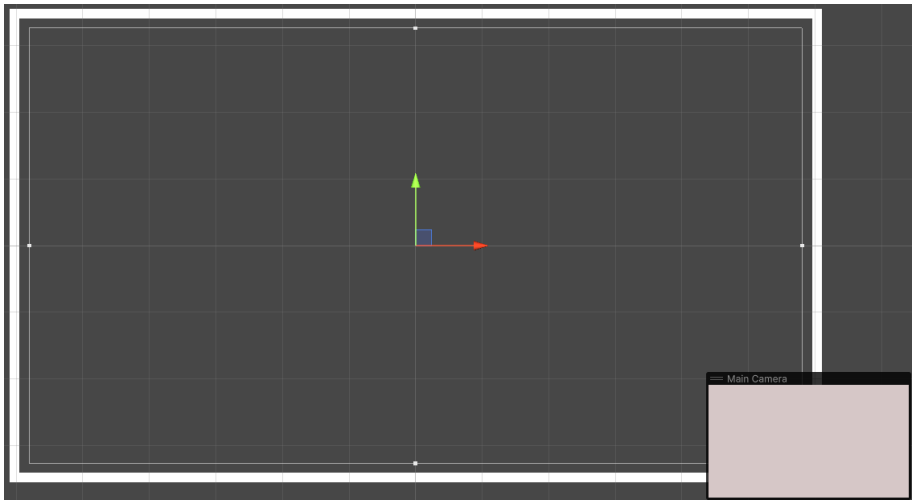


Figura 3.1: Main camera view din Unity

3.2 Mâncarea

Obiectele din ciclul de generare a mâncării sunt 2, **Food (Prefab asset)**(componente în imaginea 3.2) și **FoodManager**. **Food Prefab** reprezinta schema după care se construiește o bucata de energie. Bucățile de energie se găsesc pe harta sub forma de discuri de culoare galbena, mai exact #F6F800. Acestea, ca și pereții, reprezint obiecte statice care odată create nu se mișca. **Food Prefab** are asociate următoarele componente:

- **Circle Colider 2D** permite detectarea coliziunilor cu Obiectul
- **Rigidbody 2D** permite ca obiectul sa poată fi lovit de Raycasts
- **Food Controller** un script C# care controlează distrugerea obiectului.

Obiectele construite prin FoodPrefab au scala de 1 pe axa X, Y și Z, și sunt asociate cu tag-ul ”Food” pentru a permite detecția lor prin Raycasts și coliziuni. De asemenea, ele sunt asociate cu **”Layer”**-ul ”Food” pentru a permite detecția lor de către senzorul ”Field of View”(secțiunea 3.3).

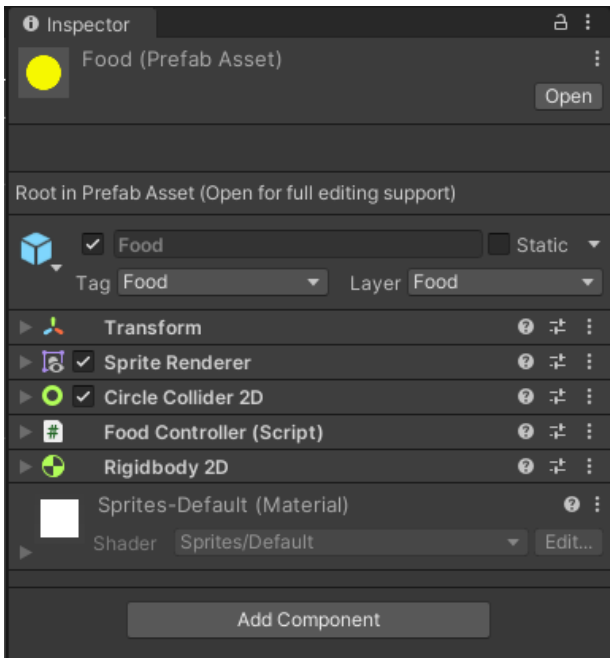


Figura 3.2: Componentele Prefab-ului Food

FoodManager este un ”Empty Object”, al cărui singur scop este sa fie un obiect permanent în aplicație de care putem atașa un script. Scriptul pentru **FoodManager** ne permite sa asociem acestuia un ”Prefab”, în cazul acesta **Food Prefab**, iar scriptul ne oferă funcții pentru a crea un obiect sau mai multe din ”Prefab-ul” asociat, și ne permite și sa le distrugem pe toate. În **Food Controller**, scriptul pentru **FoodPrefab**, funcția ”DieAndRespawn” permite distrugerea obiectului Food și regenerarea sa într-o alta locație în mediu, printr-un apel la FoodManger. Astfel putem păstra numărul de bucăți de energie constant în mediu, chiar dacă energia este colectata în continuu de speciile de prada.

3.3 Creaturile

În mediul nostru virtual avem doua tipuri de creaturi, **Prada** si **Pradator**, care pot fi construite prin intermediul **Prey (Prefab Asset)** și respectiv **Predator (Prefab Asset)**. În aceasta vom enunța în mare parte proprietățile pentru **Prey Prefab**, deoarece cele 2 prefaburi au multe elemente comune, deci pentru **Predator Prefab** vom expune doar diferențele.

Prefab-ul pentru Prey definește următoarele componente:

- **Sprite Renderer**: Definim forma creaturilor prey ca fiind un triunghi, scalat la dimensiunea de 1.5 pe axa x si Y

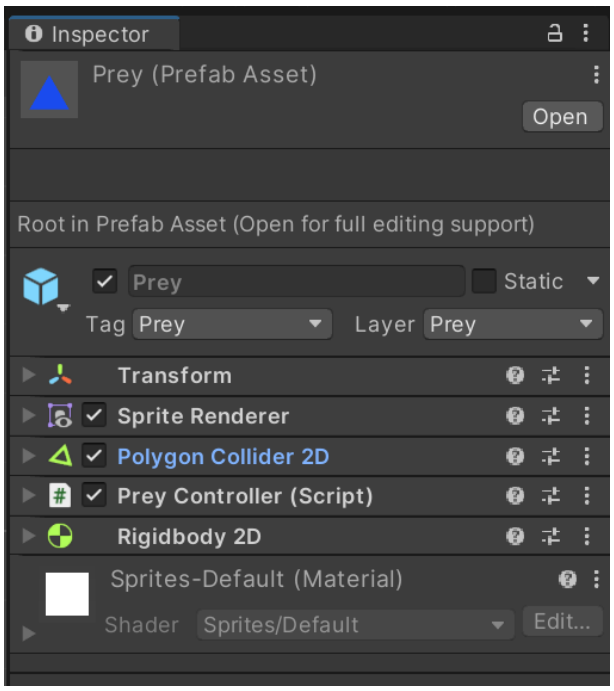


Figura 3.3: Componentele Prefab-ului Prey

- **Polygon Collider 2D**: Un Collider care ia forma sprite-ului, folosit pentru detectarea coeziunilor
- **Rigidbody 2D**: Necesar pentru detectarea coliziunilor, pentru ca detectarea coeziunilor se va face în cadrul obiectelor dinamice: Prey si Predator. Este de tip "Kinematic", spre deosebire de static pentru obiectele statice, și permite detectarea coeziunilor kineamtic/static și kinematic/kinematic.
- **Prey Controller**: script C#, moștenește clasa **Creature Controller**, care încapsulează trăsături comune atât pentru prey cat și pentru predator, definește mișcarea, senzorii, acumularea de energie și moartea obiecte Prey.

Mișcarea

Mișcarea este definita de schimbări care au loc asupra vitezei și a unghiului de rotație al creaturii. O creatura se poate deplasa doar în fata sau în spate, dar aplicând rotații asupra proprietății "transform.eulerAngles" se poate schimba direcția în care aceasta merge. Schimbările asupra vitezei și a rotației obiectului sunt determinate de o rețea neuronala asociata obiectului, care primește ca input un număr de valori senzor, și returnează doua valori în intervalul (-1, 1), prima fiind modificarea asupra accelerației iar a doua modificarea asupra rotației. Acest proces are loc în funcția "Fixe-

dUpdate()", deci la fiecare schimbare de frame stratul de intrare se încarcă cu valorile senzorilor, se executa pasul de feed-forward iar valorile din stratul de ieșire sunt folosite pentru a modifica poziția creaturii.

Senzori

În mod implicit, toate creaturile au următorii senzori:

- **Bias**: este un senzor care are mereu valoarea 1, folosit pentru a adăuga bias la nodurile care creează o conexiune cu el.
- **Perete**: acesta este un singur senzor, definit de un Raycast în fata obiectului(în planul 2D, aceasta reprezinta direcția "up" pentru proprietatea "transform" a obiectului). Un raycast reprezinta proiectarea unei raze invizibile de o anumita lungime dintr-un punct, pe o direcție in scena. Când aceasta lovește collider-ul unui alt obiect din scena, este returnat un obiect din clasa "RaycastHit2D"(pentru mediul nostru 2D), care poate fi analizat pentru a determina ce tip de obiect a fost lovit(prin intermediul tag-ului) obiectului, și distanta de la punctul de pornire la care a fost lovit. Pentru senzorul de pereți, dacă raycast-ul lovește un perete, valoarea sa va fi raportul dintre distanta pana la lovitura și lungimea razei(deci, intre 0 și 1). Dacă nu a fost detectat un perete, valoarea senzorului e 1.
- **Viteza curenta**: valoarea senzorului este valoarea precedenta a ieșirii pentru "viteza" a rețelei neuronale.
- **Unghi**: unghiul curent, exprimat intre -1 și 1, dintre direcția înainte a creaturii și axa Y a scenei.

Pentru detecția țintelor creaturilor(adică a surselor de energie pentru fiecare tip în parte, prădătorii detectează creaturi de tipul prada, iar prada detectează bucățile de energie), am folosit în timp 2 tipuri de senzori. Cel mai simplu reprezinta un sistem format din 3 Raycast-uri, o sa îl numim **fork** pentru a ne referi la el mai departe, care acționează la fel ca senzorul implicit pentru perete, doar ca în loc sa detecteze pereți, detectează țintele creaturii. Un raycast este proiectat pe direcția înainte a obiectului, iar celelalte doua la un unghi de $45°$ respectiv $-45°$ fata de aceeași direcție.
Ulterior am dezvoltat un senzor mai complex, de tip "Field of View", al cărui concept vine din [SBM05], secțiunea 5, figure 10, unde sunt descriși mai multi senzori printre

care și ”Enemy Radars”. Acesta descrie divizarea spațiului de $360°$ din jurul agentului în mai multe zone. Fiecare zona activează un senzor în dependenta de cat de apropiat este un inamic de agent. Dacă sunt mai multi inamici în aceeași zona, se sumează activările pentru fiecare inamic în parte.

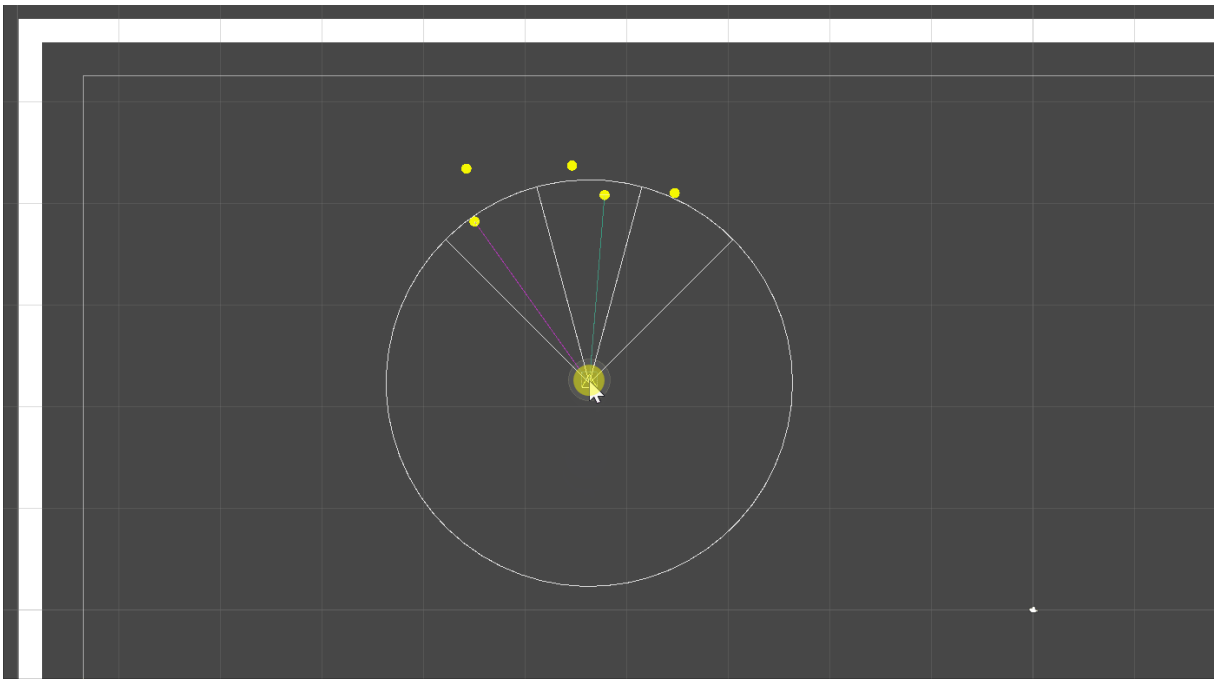


Figura 3.4: Senzorul FoV si senzorul pentru perete din cadrul prazii, liniile colorate arata distnata pana la fragmentele de energie care sunt detectate de diferitele zone ale senzorului. Linia alba atunci cand prada se apropie de perete este raycast-ul catre perete

În cadrul implementării noastre, pentru senzorul ”Field of View”(**FoV**), alegem un unghi, intre 0 și 180 de grade, care reprezinta câmpul vizual din fata unei creaturi, și raza pana la care acesta se extinde. Prin alt parametru, specificam în cate zone se împarte acesta. Prin intermediul metodei ”Physics2D.OverlapCircleAll”, se detectează toate țintele din cadrul unui cerc din jurul creaturii, după care, analizând unghiul dintre direcția înainte a creaturii și poziția unei ținte, putem determina dacă țintele cad în cadrul câmpului vizual, și în ce zona a câmpului. Pentru fiecare zona, activarea ei va reprezenta valoarea calculului $\sum_i 1 - d_i/r$, i fiind o ținta din cadrul zonei, d_i distanta pana la ținta, iar r raza cercului în care are loc detecția. Deci pentru fiecare ținta avem o activarea intre 0 și 1, care este mai mare cu cat ținta e mai aproape de creatura. În cadrul animației 3.4, se poate vedea cum funcționează senzorul ”FoW”, cu un câmp vizual de $90°$, împărțit în 3 zone, pe o distanta de 20 ”world units”, iar la sfârșitul

animație putem vedea și o vizualizare a raycast-ului pentru senzorul de perete. Pentru a scrie vizualizarea în ”Editor View” a senzorului ”FoW” un mare ajutor a fost [Lag15]. În unul din experimentele de la capitolul 6, vom arata ca prada care folosește senzorul ”FoW” este superioara celei care folosește ”Fork”, și vom prezenta și diferențele de comportament intre cele 2 tipuri de prada. Pentru prădătorii la senzori nu se schimba nimic decât țintele căutate, acestea fiind creaturile de prada.

Coliziuni

Datorita faptului ca toate obiectele care interacționează în cadrul scenei au asociat cate un collider și un rigidbody, putem detecta coliziunile intre ele. Procesare acestor coliziuni se face în cadrul script-uri-lor ”Prey Controller” și ”Predator Controller”, asociate prefab-ului ”Prey” și respectiv ”Predator”. Prin intermediul tag-urilor ”Wall” și ”Food”, script-ul ”Prey Controller” înregistrează coliziunile cu pereții, ceea ce cauzează moartea prăzii, și coliziunea cu fragmentele de energie, prin care energia asociata fragmentului este absorbita, iar fragmentul prin funcția ”DieAndRespawn” este regenerat la alte coordonate aleatorii în cadrul scenei, deci cantitatea de energie în scena este constanta. Procesare coliziunilor prada-prădător are loc în script-ul ”Predator Controller”, și pot fi identificate prin tag-ul asociat creaturilor de prada, ”Prey”. În cadrul acestor coliziuni, obiectul prada este distrus, însa obiectul prădător primește energie, într-o cantitate semnificativ mai mare decât ce primesc creaturile prada la coliziunile prada-fragment de energie, plus un procent din energia acumulata de prada de-a lungul vieții. Facem aceste ajustări deoarece vânatul pentru prada este mult mai ușor decât cel pentru prădător,pentru ca prădătorii trebuie sa dezvolte comportamente de chassing, astfel ca prădătorii ar trebui sa primească mai multa energie din vânat pentru a ne asigura ca aceștia nu sunt surclasați de prada în cadrul evoluției. Coeziunile prădător-perete sunt la fel ca la prada, însa prădătorii nu înregistrează coliziunile cu fragmentele de energie, deoarece vrem ca aceștia sa obțină energie doar prin coliziunile cu prada. Pe tot parcursul vieții lor, creaturile consuma energie în mod constant, plus o cantitate care depinde de viteza cu care se mișca. Astfel le încurajam sa acumuleze energie încontinuu. Odată ce o creatura rămâne fără energie, aceasta moare.

3.4 Managementul Algoritmului

NeatManager, ca și FoodManger, este un "Empty Object" pe care îl folosim pentru a atașa script-ul "Neat Manager". Acest script este responsabil de desfășurare algoritmului Neat, deci are ca responsabilități generarea populației inițiale, înregistrarea morților, crearea generațiilor următoare și înregistrarea de log-uri despre populație pe parcursul generațiilor. În cadrul NeatManager putem decide sa efectuam experimente în care folosim doar creaturile de prada, sau și prădătorii, și putem determina parametri cum ar fi numărul de membri, numărul de noduri pentru stratul de intrare cat și cel de ieșire etc.(vom discuta mai pe larg mulțimea de parametri în secțiunea 5). Faza de generare a populației inițiale se desfășoară în următorul mod:

1. Se generează genomul inițial pentru fiecare membru din populație(reprezinta o codificare Neat a unei rețele cu structura minima dar cu ponderi ale conexiunilor aleatorii, revedeți secțiunea **??**)

2. Se generează fenotipul pentru fiecare genotip(rețeaua neuronala propriu-zisa, vom expune maparea de la fenotip la genotip în capitolul 4)

3. Se creează agenții simulării, După prefab-ul care a fost asignat către NeatManager. Proprietățile acestora sunt setate După parametrii NeatManager pentru un prefab specific, și sunt poziționați la coordonate aleatorii în cadrul "Main Camera View", fiecăruia dintre ei fiindu-i asignata și una dintre rețele create la pasul precedent.

După creare, agenții sunt liberi sa exploreze mediul și sa acumuleze fitness. În secțiunea anterioara nu am menționat ca în toate scenariile în care un agent moare, asta fiind valabil atât pentru prada cat și pentru prădători, acesta își înregistrează moartea și cauza ei la "NeatManager", prin intermediul funcției "RegisterDeath". Astfel, NeatManager poate determina când toți agenții din mediu au fost distruși, și poate începe crearea următoarei generații. Înainte de crearea următoarei generații, se log-eaza fitness-ul average și fitness-ul celui mai bun agent din generația curenta, plus informații despre structura celui mai bun agent. Procesul de generare a următoarei generații decurge la fel ca cel de inițializare, doar ca mai întâi se aplica o filtrare pentru a rămâne doar genom-urile cele mai bune din cadrul fiecărei specii, care vor trece prin procesul de crossover și de mutație pentru generarea genomurilor generației viitoare. Aceste genom-uri sunt asignate către diferite specii din populație, care grupează genomuri

similare. După acest pas se creează noile rețele neuronale, și se generează noii agenți care gândesc prin intermediul acestora, iar tot procesul se repeta din nou la fiecare generație. Ne-am permis în aceasta secțiune sa facem un rezumat al funcționarii algoritmului, vorbind despre obiectul care este responsabil de desfășurarea lui, însa în capitolul 4 vom expune detaliat, cu exemple din cod, procesele din cadrul algoritmului.

Capitolul 4

Detalii de implementare

În cadrul algoritmului NEAT, expus în [SM02] sunt expuse în mod teoretic inovațiile aduse de NEAT ca algoritm de Neuro-Evolutie, însa nu avem o implementare a lor, cum ar fi, de exemplu, generarea numerelor de inovație. În acest capitol vom concretiza aspecte puțin expuse în NEAT, aspecte care țin doar de de simulare, precum și îmbunătățiri sau schimbări propuse mai târziu pe "The NEAT Users Page"[Sta15], în secțiunea "Neat Software FAQ".

4.1 Genomul în NEAT

Structura genomului în NEAT este foarte simpla, și intuitiv de implementat, la fel și operațiile din cadrul mutației, care implica perturbarea ponderilor conexiunilor, resetare ponderilor cu noi valori aleatorii, adăugarea de noi noduri și adăugarea de conexiuni, descrise bine în "Figure 3" din [SM02]. În cadrul operației de crossover, de asemenea, prin intermediul fitnessului este ușor sa înțelegem genele cărui părinte vor fi moștenite de copil. Totuși, pentru operația de aliniere a genom-urilor părinților, exemplul din "Figure 4" din [SM02] nu este destul de bun, deoarece el ascunde complexitatea operației în cazul în care genele matching nu sunt toate la început-ul fiecăruia dintre părinți, ci se pot găsi și printre genele "disjoint".

Listing 4.1: Parcurgerea genom-urilor celor 2 parinti pentru alinierea lor in cadrul crossover. Prin "this" sunt accesate campurile primului parinte

```
1    int idx1 = 0, idx2 = 0;
2    while (idx1 < this.connectionGenes.Count && idx2 < parent2.
         connectionGenes.Count)
3    {
```

```
        var p1Gene = this.connectionGenes[idx1];
        var p2Gene = parent2.connectionGenes[idx2];
        if (p1Gene.inovationNr == p2Gene.inovationNr)
        {
            child.AddMatchingGene(p1Gene, p2Gene, inheritDisabledGeneProb);
            idx1++;
            idx2++;
        }
        else if (p1Gene.inovationNr < p2Gene.inovationNr)
        {
            if (p1Fitness >= p2Fitness)
            {
                child.AddCopyOfGene(p1Gene);
            }
            idx1++;
        }
        else
        {
            if (p2Fitness > p1Fitness)
            {
                child.AddCopyOfGene(p2Gene);
            }
            idx2++;
        }
    }

    if (idx1 < this.connectionGenes.Count && p1Fitness >= p2Fitness)
    {
        for (int i = idx1; i < this.connectionGenes.Count; i++)
        {
            child.AddCopyOfGene(this.connectionGenes[i]);
        }
    }
    else if (idx2 < parent2.connectionGenes.Count && p2Fitness > p1Fitness)
    {
        for (int i = idx2; i < parent2.connectionGenes.Count; i++)
        {
            child.AddCopyOfGene(parent2.connectionGenes[i]);
        }
    }
```

Soluția noastră, expusa în 4.1, propune folosirea a 2 indexuri pentru parcurgerea ambelor genomuri în același timp. Dacă pentru poziția curenta a indexurilor ambele gene au numere de inovație egale, avansam ambele indexuri, dacă părintele 1 are numărul de inovație mai mare, avansam indexul părintelui 2, deoarece este posibil ca la următoarea poziție genele sa fie matching, iar altfel, avansam index-ul pentru părintele 1. Dacă ajungem la sfârșit și genomul unui părinte sa terminat înaintea celuilalt, înseamna ca restul genelor din părintele care nu sa epuizat sunt în exces. Aceeași aliniere o facem și în cadrul operației de calcul al distante δ dintre 2 genom-uri, doar ca în loc sa asignam gena unui sau a altui părinte la copilul ce se formează, contorizam genele ”matching”, cele disjoint și cele ”excess”.

4.2 Rețelele neuronale din NEAT

Rețelele neuronale din NEAT reprezinta rețele neuronale recurente. Acestea au cicluri care pot porni din straturile adânci ale rețelei spre cele din fata, cum ar fi de la cele de intrare la cele de ieșire, și nu este garantat ca fiecare nod dintr-un strat va fi conectat la fiecare nod din următorul, deci reprezentarea și evaluarea din cadrul rețelelor dense feed-forward nu poate fi aplicata aici. Lucrarea [SM02] nu are specificații pentru generarea rețelelor, în afara de faptul ca genele de conexiune setate ca disabled nu ar trebui sa fie exprimate în fenotip, adică în rețea.

Listing 4.2: Structura pe clase a unei retele NEAT

```
public class NeatNetwork
{
    public float fitness;
    public int index;
    private enum CycleDetection { Unverified, Verified, Cycle }
    private readonly List<int> evalOrder;
    private  List<int> inputs;
    private  List<int> outputs;
    private readonly Dictionary<int, Node> nodes;
    ...
}

internal class Node
{
    public int id;
```

```
    public float value;
    public List<Connection> inputConnections;
    public List<Connection> outputConnections;
    ...
}

internal class Connection
{
    public int inNode;
    public int outNode;
    public float weight;
    ...
}
```

O rețea, după cum am listat în 4.2, are următoarele componente:

- **fitness**: stochează fitness-ul unui agent din populație odată cu moartea sa.
- **index**: un id al rețelei asignat de către NeatManager.
- **nodes**: un dicționar constituit din perechi de id-uri ale nodurilor și noduri. Conține toate nodurile necesare rețelei, iar fiecare nod are propriile liste de conexiuni cu alte noduri.
- **evalOrder**: o lista de id-uri de noduri, reprezentând ordinea în care nodurile ar trebui evaluate în rețea pentru a obține un output corect.
- **inputs și outputs**: liste cu id-urile nodurilor de intrare, respectiv de ieșire.
- **CycleDetection**: un enum ce va fi folosit pentru stabilirea conexiunilor care formează un ciclu în rețea.

Prin stabilirea unei ordini de evaluare a nodurilor din rețea la crearea rețelei, evaluarea va putea avea loc simplu și rapid, ceea ce e importanta deoarece o rețea va fi evaluata de foarte multe ori pe parcursul vieții sale, la fiecare frame al simulării. Pentru generare unei rețele RNN din genotip, vom trece prin 4 pași:

1. Filtrarea conexiunilor enabled si crearea nodurilor necesare pentru conexiuni.
2. Detectarea ciclurilor din cadrul rețelei, ne referim la conexiunile care completează un ciclu.

3. Eliminarea structurilor care nu sunt conectate la nici un nod de intrare, acestea nu primesc date de la senzori, deci sunt redundante și nu aduc nici un beneficiu rețelei.

4. Stabilire unei ordini de evaluare a nodurilor, cu ajutorul informației despre cicluri pe care o avem de la pasul 2.

Pentru prima rutina, întâi cream nodurile care corespund straturilor de intrare și celor de ieșire pe care le adaugăm în `nodes`. Ulterior, pentru fiecare "connection gene" care nu este dezactivata, verificam dacă nodul de intrare și cel de ieșire sunt prezente în `nodes`, altfel, le cream și le adaugăm. Prin informația din "connection gene", putem construi un nou obiect "Connection", iar aceasta conexiune o adaugăm în lista de `outputConnections` pentru nodul de intrare, și cea de `inputConnections` pentru nodul de ieșire. Astfel, fiecare nod știe cu ce alte noduri este conectat, ponderea conexiunilor și direcția conexiunilor. Păstram și o lista a tuturor conexiunilor create care va fi folosita la pasul următor.
În pasul 2, pentru determinarea nodurilor ce formează cicluri, folosim algoritmul 4.3.

Listing 4.3: Algoritmul de detectare a conexiunilor ce formeaza cicluri

```
1  private Dictionary<Connection, (CycleDetection verification, bool mark)>
       DetectCycles(List<Connection> connections)
2  {
3      Dictionary<Connection, (CycleDetection verification, bool mark)>
           verificationDictionary = new();
4      connections.ForEach(conn => verificationDictionary.Add(conn, (
           CycleDetection.Unverified, false)));
5      foreach (var nodeId in inputs)
6      {
7          var currRoute = new List<int>() { nodeId };
8          ExploreRouteCiclicity(currRoute, verificationDictionary);
9      }
10
11     return verificationDictionary;
12 }
13
14 private void ExploreRouteCiclicity(List<int> route, Dictionary<Connection,
       (CycleDetection verification, bool mark)> verificationDictionary)
15 {
16     foreach (var conn in nodes[route[^1]].outputConnections)
```

```
17      {
18          if (verificationDictionary[conn].verification == CycleDetection.
                Unverified)
19          {
20              if (route.Contains(conn.outNode))
21              {
22                  verificationDictionary[conn] = (CycleDetection.Cycle, false
                        );
23              }
24              else
25              {
26                  verificationDictionary[conn] = (CycleDetection.Verified,
                        false);
27                  ExploreRouteCiclicity(new List<int>(route) { conn.outNode }
                        , verificationDictionary);
28              }
29          }
30      }
31  }
```

Ideea este sa inițializam un dicționar unde cheile sunt conexiunile, deoarece ele sunt unice, iar valorile sunt statusul verificării conexiunii(inițial toate conexiunile sunt marcate ca fiind unverified), plus un marcaj ce va fi folosit la pasul 4. După care, pentru fiecare nod de intrare, începem o căutare "Depth-first search"(DFS). Practic exploram toate conexiunile de ieșire a nodului curent, iar pentru conexiunile care sunt marcate ca neverificate, dacă nodul de ieșire al conexiunii este deja prezent în ruta pe care am parcurso pentru a ajunge la nodul curent, însemna ca am găsit un ciclu, și marcam conexiunea în acest sens, altfel, marcam conexiunea ca fiind verificata, iar nodul de ieșire al conexiunii este adăugat în ruta ca fiind nodul curent și continuam explorarea. Totuși, dacă analizam un exemplu de rețea cum ar fi cel din imaginea 4.1, vedem ca nu toate conexiunile sunt verificate. Verificările încep de la fiecare nod din stratul de intrare în parte, deci o conexiune care rămâne neverificata este una la care nu ajung prin nici o cale informațiile din stratul de intrare. Astfel, structurile neconectate pot fi șterse fără sa afecteze în nici un mod evaluarea rețelei, ci chiar facilitează procesul de stabilire a ordinii de evaluare a nodurilor. Procesul de eliminare al structurilor redundante este simplu: fiecare conexiune rămasa în starea unverified va fi ștearsa din dicționarul de verificări, și din listele nodurilor pe care le conectează, după care este

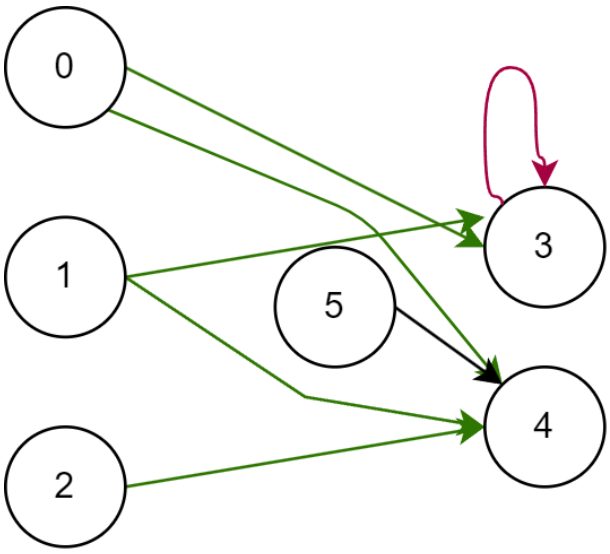


Figura 4.1: Un exmplu de retea dupa verificarea ciclicitatii conexiunilor. Sagetile verzi sunt conexiunile cu starea de verified, cele rosii au starea de cycle, insa putem vedea ca conexiunea 5-4 ramane unverified(sageata neagra), datorita faptului ca nu este conectata la nici un input.

parcurs dicționarul `nodes` și sunt eliminate toate nodurile ascunse care nu au nici o conexiune de input. Ultima parte, stabilirea ordinii de evaluare a nodurilor este expusa în listing-ul 4.4. Logica acestei operații este următoare: un nod poate fi evaluat odată ce au fost evaluate toate nodurile cu care acesta este conectat prin lista de conexiuni `inputConnections`, cu excepția nodurilor de intrare ce sunt conectate de nodul curent prin conexiuni ciclu, deoarece de la aceste nodul curent va lua valoarea lor de la evaluarea anterioara. Astfel, la început, aceasta condiție este îndeplinita doar de o parte din nodurile de intrare, care nu au `inputConnections` sau care au doar cicluri. Definim o lista, `currOrder` în care punem nodurile care indeplinesc aceasta condiție, iar într-o bucla, pana când toate nodurile sunt incluse în `evalOrder`:

1. Cu ajutorul `verificationDictionary`, pentru fiecare nod din `currOrder`, schimbam marcajul conexiunilor sale de ieșire din $false$ în $true$.

2. Cream o lista goala next.

3. Pentru fiecare nod care a rămas sa fie asignat ordinii: dacă toate conexiunile sale de intrare sunt cicluri sau sunt marcate true, asta însemnând ca nodul de intrare indicat de conexiune a fost deja adăugat în ordinea de evaluare. Deci, în timpul evaluării valoarea sa ar fi deja calculata, nodul curent poate fi adăugat în lista next.

4. Adaugăm la `evalOrder` nodurile din `currOrder`, iar noul `currOrder` va fi lista next.

Astfel, gradual, la fiecare iterație a buclei, nodurile care pot fi evaluate cu informația valabila la moment for fi adăugate în ordinea de evaluare, pana când acesta este completata cu toate nodurile rețelei.

Listing 4.4: Functia de determinare a ordinii de evaluare a nodurilor

```
private void EstablishEvalOrder(Dictionary<Connection, (CycleDetection
    verification, bool mark)> verificationDictionary)
{
    var remaining = nodes.Keys.ToList();

    var currOrder = inputs.Where(nodeId => nodes[nodeId].inputConnections.
        All(conn => verificationDictionary[conn].verification ==
        CycleDetection.Cycle)).ToList();
    foreach (var node in currOrder)
    {
        remaining.Remove(node);
    }

    while (remaining.Count > 0)
    {
        foreach (var nodeId in currOrder)
        {
            foreach (var con in nodes[nodeId].outputConnections)
            {
                if (verificationDictionary[con].verification ==
                    CycleDetection.Verified)
                {
                    verificationDictionary[con] = (CycleDetection.Verified,
                        true);
                }
            }
        }

        var next = new List<int>();
        for(int i = 0; i < remaining.Count; i++)
        {
            var nodeId = remaining[i];
            if (nodes[nodeId].inputConnections.All(conn =>
```

```
                verificationDictionary[conn].mark || verificationDictionary[
                conn].verification == CycleDetection.Cycle))
            {
                next.Add(nodeId);
                remaining.RemoveAt(i);
                i--;
            }
        }

        evalOrder.AddRange(currOrder);
        currOrder = next;
    }

    evalOrder.AddRange(currOrder);
}
```

4.3 Algoritmul NEAT

Responsabil de execuția algoritmului NEAT în cadrul proiectului nostru este script-ul ”NeatManager.cs” atașat obiectului NeatManager din scena. Cea mai mare parte a algoritmului este integrata în cadrul funcției `ProduceNextGeneration()` (De văzut listing-ul 4.5)

Listing 4.5: Generarea urmatoarei generatii

```
private void ProduceNextGeneration()
{
    //inovationManger.ResetForNextGeneration();
    AdjustCompatibiltyTreshold();
    ComputeAllSpeciesAdjustedFitness();
    AdjustAllSpeciesPotentialMemeberCount();
    GenerateNextGenGenomes();
    SpeciesRepartition();
    GenerateNetworks();
    GenerateCreatures();
    currentGeneration++;
}
```

Se poate observa ca prima linie este comentata: de ce? În cadrul implementării noastre, `InovationManger` este o clasa a cărei obiecte ne permit sa ținem cont de

inovațiile ce apar în genom-urile populație în urma mutațiilor structurale. În pagina [Sta15], este adresata întrebarea dacă ar trebuie sa păstram înregistrările noilor inovații doar pentru generația curenta sau pe tot parcursul algoritmului. Ca răspuns Keneth O. Stanley precizează ca efectul principal al păstrării numerelor de inovație pe tot parcursul algoritmului este ca se vor genera mai puține specii, însa acesta nu a definit o metoda ca fiind mai buna decât alta, asta depinzând de experiment. În cadrul simulatorului nostru totuși, păstrarea permanenta a numerelor de inovație a produs rezultate mai bune, din acest motiv linia 3 este comentata, pentru a nu reseta înregistrările la fiecare generație.

Funcția `AdjustCompatibiltyTreshold` de asemenea este inspirata din unul din răspunsurile de pe pagina [Sta15], anume dacă exista o metoda pentru selecta în mod automat pragul de compatibilitate δ_t pentru speciație. Aceasta metoda implica setare unui număr ținta de specii în cadrul algoritmului, iar acest pragul de compatibilitate va scădea dacă avem mai puține specii decât numărul ținta, sau va creste dacă sunt prea multe specii. În cadrul implementării proprii am setat un minim și un maxim pentru acest prag cat și o valoare prin care pragul se modifica la începutul fiecărei generații(vedeți 4.6).

Listing 4.6: Ajustarea pragului de compatibilitate

```
1  private void AdjustCompatibiltyTreshold()
2  {
3      if (currentGeneration > 1)
4      {
5          if (species.Count < speciesTargetCount && compatibilityTreshold >
               minCompatibilityTreshold)
6              compatibilityTreshold -= compatibilityModif;
7          else if (species.Count > speciesTargetCount &&
               compatibilityTreshold < maxCompatibilityTreshold)
8              compatibilityTreshold += compatibilityModif;
9
10         if (compatibilityTreshold < minCompatibilityTreshold)
               compatibilityTreshold = minCompatibilityTreshold;
11         if (compatibilityTreshold > maxCompatibilityTreshold)
               compatibilityTreshold = maxCompatibilityTreshold;
12     }
13 }
```

Fiecare membru din populație aparține unei specii. Apartenența la o anumita

specie e determinata de genomul membrului. O specie în cadrul implementării noastre are următoarea structura:

Listing 4.7: Reprezentarea unei specii in algoritm

```
public class Species
{
    public int Id { get; set; }
    public List<int> MembersIdx { get; set; }
    public NeatGenome SpeciesRepresentative { get; set; }
    public int PotentialMembersCount { get; set; }
    public float LastBestFitness { get; set; }
    public int GenerationsSinceImprovement { get; set; }
    public int GenerationsSinceMemebersAssigned { get; set; }
    public float SpeciesAdjustedFitness { get; set; }
    public (byte r, byte g, byte b) RgbColor { get; set; }
    ...
}
```

Vom descrie aceste proprietăți în cadrul metodelor care le accesează.

În `ComputeAllSpeciesAdjustedFitness()` se întâmpla următoarele operațiuni: pentru fiecare specie, membrii din `MembersIdx` sunt sortați după fitness-ul rețelelor lor si se determina dacă exista o îmbunătățire a celui mai bun fitness al speciei de pana acum accesând `LastBestFitness`. În caz contrar, se incrementează counter-ul `GenerationsSinceImprovement`, ceea ce duce la eliminarea speciei dacă se contactează ca aceasta a stagnat de prea multe generații. Mai apoi, se calculează media fitness-ului membrilor speciei, ce este asignata proprietății `SpeciesAdjustedFitness`. Aceasta medie va fi folosită în următoarea funcție, `AdjustAllSpeciesPotentialMemeberCou` unde se calculează, ca un raport intre fitness-ul mediu al speciei și suma fitness-urilor medii ale tuturor speciilor, ce procent din populație va genera potențiali membri ai speciei. În acest mod se calculează numărul de membri potențiali, `PotentialMembersCount`. Tot în cadrul acestei funcții eliminam și membrii speciei curente care au performante mai proaste. De exemplu dacă păstram doar 30% din membri, ceilalți 70% sunt eliminați. Numărul de membri potențiali pentru fiecare specie este folosit în metoda `AdjustAllSpeciesPo` pentru a determina de cate ori vom utiliza o specie pentru alegerea în mod aleatoriu ai părinților unui nou copil, al cărui genom e generat prin crossover și mutație în cadrul metodei `GenerateNextGenGenomes`. Tot in aceasta metoda pentru fiecare specie existenta va fi ales in mod aleator dintre membrii sai un reprezentant, inainte

initierii procesului de creare de noi copii care vor inlocui populatia curenta. Se poate remarca ca operatorii genetici pot aduce schimbări asupra genomului copilului astfel încât acesta nu se mai încadrează în nici una din speciile părinților sai (în cazul în care are loc crossover intre 2 indivizi din specii diferite), însa adoptând o sugestie din cadrul [Sta15] pentru a mari viteza cu care are loc specierea, copilul va primi ca `speciesHint` proprietatea `Id` a speciei primului părinte.

Metoda `SpeciesRepartition()` se ocupa de repartizarea genom-urilor noi create la pasul anterior la cate o specie. Pentru fiecare genom care reprezinta noua populație, determinam prin intermediul proprietății sale `speciesHint`, dacă acesta aparține același specii ca părintele său. Aceasta informație se afla calculând distanta de compatibilitate δ dintre `SpeciesRepresentative` si genom-ul curent. Daca δ nu este mai mic decat pragul de compatibilitate, adica genomul copilului nu apartine speciei parintelui, refacem acelasi calcul cu fiecare dintre celelalte specii. Dacă genomul curent nu este compatibil cu niciuna dintre speciile din populație, o noua specie este creata, aceasta primește un nou `Id` unic, o culoare pe care o generam pe baza id-ului(aceasta va fi culoarea obiectelor din scena care reprezinta membri ai speciei), reprezentantul speciei va fi genomul curent, și tot el va fi și unicul membru, dacă nu sunt asignați și alte genom-uri acestei specii.

În cadrul acestei metode vom executa și operația de eliminare a speciilor redundante. Acestea sunt specii care nu au avut asignat nici un membru timp de mai multe generații, valoare care o putem accesa prin prorietatea `GenerationsSinceMemebersAssigned` a speciei. Nu eliminam specia încă de la prima generație în care nu i-a fost asociat nici un membru, deoarece e posibil ca în timpul unei generații ulterioare acesteia sa ii fie asociat unul, deci evitam crearea speciilor noi fără motiv.

Fiindcă toate genom-urile populației noi au fost asociate speciilor cu care sunt compatibile, tot ce mai rămâne de făcut sa generăm din genom rețeaua neuronala pentru fiecare individ din populație, după cum am descris în secțiunea **??**, prin metoda `GenerateNetworks()`, și sa generam creaturile din scena care vor folosi aceste rețele prin metoda `GenerateCreatures()` pe care o expunem în listing-ul 4.8 ca un exemplu pentru cum arata generarea unei populații de prada, cu tot cu parametri.

Listing 4.8: Generarea speciilor de prada

```
1 var foodMask = LayerMask.GetMask("Food");
2 foreach (var s in species)
3 {
```

```
    foreach (int member in s.MembersIdx)
    {
        var pos = new Vector2(UnityEngine.Random.value, UnityEngine.Random.
            value);
        pos = Camera.main.ViewportToWorldPoint(pos);

        allCreatures[member] = Instantiate(creaturePrefab, pos, transform.
            rotation);
        var creatureController = allCreatures[member].gameObject.
            GetComponent<PreyController>();
        currentPopulation++;

        creatureController.network = allNetworks[member];
        creatureController.inputNodes = inputNodes;
        creatureController.outputNodes = outputNodes;
        creatureController.totalEnergy = startingEnergy;
        creatureController.energyForFood = energyForFood;

        creatureController.fowTargetsMask = foodMask;
        creatureController.wallRayDistance = 20f;
        creatureController.viewDistance = 20f;
        creatureController.viewAngle = 90f;
        creatureController.angleSplits = 3;
        (var r, var g, var b) = s.RgbColor;
        creatureController.spriteColor = new Color32(r, g, b, 255);
    }
}
```

Capitolul 5

Parametrii experimentali

In acest capitol vom descrie in primul rand parametrii tuturor obiectelor din cadrul scenei care pot influenta simularea. Ulterior vom descrie setul de simulari, scopul fiecareia, si setul de parametri care le definesc.

5.1 Main Camera si Walls

- `Size` : schimba marimea GameView-ului prezentat de camera, si deci schimba si marimea zonei in care se genereaza fragmente de energie si creaturi.
- `Wall transform`: peretii doar trebuie scalati si repozitionati astfel incat sa inchida complet spatiul din interior, dar sa fie putin inafara view-ului camerei principale.

5.2 FoodManager

- `Food Prefab`: prefab-ul obiectului ce va fi folosit pe post de mancare
- `Total Food`: numar intreg ce reprezinta cate obiect Food Prefab vor fi generate si pozitionate in scena. Acest numar ramane constant pe parcursul unei simulari.

5.3 Prey si Predator

- `Acceleration Factor`: un factor ce scaleaza viteza unei creaturi, care in mod implicit e intre -1 si 1.

- `Turn Factor`: cu cate grade se poate roti o creatura intr-o directie.
- `Smoothing Factor`: factor ce afecteaza fluiditatea miscarilor, atat a rotatiilor cat si a deplasarilor inainte si inapoi.
- `Network`: reteau atribuita creaturii.
- `InputNodes`: numarul de noduri din stratul de intrarea al retelei.
- `OutputNodes`: numarul de noduri din stratul de iesire al retelei.
- `Total Energy`: cantitate de energie atribuita creaturii initial.
- `Energy For Food`: cata energie este obtinuta la coliziunea cu o sursa de mancare.
- `Speed Consumption Multiplier`: un factor ce scaleaza consumul de energie al creaturii in dependenta de viteza cu care aceasta se misca.
- `View Distance`: parametru al senzorului ”FoW”, stabileste raza din jurul creaturii in care se pot detecta tintele.
- `View Angle`: parametru al ”FoW”, stabileste unghiul din fata creaturii in care sunt detecatate tintele.
- `Angle Splits`: parametru al ”FoW”, stabileste in cate zone egale de detectie este impartit **View Angle**.
- `Fow Targets Mask`: parametru al ”FoW”, stabileste pe ce Layer va avea loc detectia, adica care sunt tintele senzorului.
- `Wall Ray Distance`: parametru al senzorului de detectie a peretelui, stabileste distanta pe care este proiectat Raycast-ul.
- `Sprite Color`: culoarea sprite-ului atasat creaturii.

5.4 NeatManager

- `Creature Prefab`: Prefab-ul din care se vor construi membrii popoulatiei
- `Input Nodes`: numarul de noduri pentru stratul de intrare al retelelor

- `Output Nodes`: numarul de noduri pentru stratul de intrare al retelelor
- `Init Weight Range`: valoare x care defineste intervalul $[-x, x]$, intre care vor fi initializate ponderile retelelor.
- `Mutation Prob`: probabilitatea ca un genom sa sufere mutatii la aplicare operatorului de mutatie.
- `Weight Perturbance Prob`: probabilitate p a fiecarei gene de conexiune din genom de a suferi o perturbare. Probabilitate ca unei ponderi sa ii fie reatribuite o valoare aleatorie e $1 - p$.
- `Perturbance Range`: valoare x care defineste intervalul $[-x, x]$, intre care se alege o valoare aleatorie pentru perturbarea unei ponderi.
- `Reassignment Range`: valoare x care defineste intervalul $[-x, x]$, intre care se alege o valoare aleatorie folosita pentru resetarea ponderii.
- `New Node Prob`: probabilitatea ca unui genom sa ii fie aplicat mutatia structurala de adaugare a unui nou nod.
- `New Connection Prob`: probabilitatea ca unui genom sa ii fie aplicat mutatia structurala de adaugare a unei noi conexiuni.
- `Inherid Disabled Gene Prob`: probabilitatea in cadrul crossover, daca una dintre doua gene aliniate ”matching” este dezactivat, copilul va mosteni o gena dezactivata.
- `Chance Mutation No Crossover`: probabilitate ca un nou membru din populatie sa nu fie generat prin crossover, ci doar prin operatorul de mutatie aplicat nodului parinte
- `Inter Species Mating Rate`: probabilitate in timpul ca operatii de crossover ca al doilea parinte sa fie ales dintr-o specie aleatorie
- `Species Dropoff Age`: numarul de generatii pe care o specie il poate supravetui daca aceasta nu prezinta imbunatatiri ale fitness-ului celui mai bun membru.
- `Survival Treshold`: procentul din membrii cei mai buni ai unei specii care sunt pastrati pentru crearea genom-urilor populatiei din generatia viitoare.

- `Species Target Count`: Numarul tinta de specii in cadrul populatiei.
- `Compatibility Treshold`: pragul de compatibilitate δ_t. Daca distanta de compatibilitate dintre doua genom-uri este mai mare decat aceasta valoare, atunci genom-urile fac parte din specii diferite.
- `Min Compatibility Treshold`: valoarea minima pentru pragul de compatibilitate
- `Max Compatibility Treshold`: valoarea maxima a pragului de compatibilitate
- `Compatibility Modif`: valoare cu care este modificat pragul de compatibilitate, daca suntem sub numarul tinta de specii, δ_t va scadea cu atat la fiecare generatie, iar daca suntem peste numarul tinta, δ_t va creste cu atat.
- `Excess Coef`: valoarea $c1$ din formula 1.1
- `Disjoint Coef`: valoarea $c2$ din formula 1.1
- `Weight Diff Coef`: valoarea $c3$ din formula 1.1
- `Top Members Selection Cutoff`: daca o specie are mai mult de atatia membri, atunci genomul celui mai bun membru al speciei va trece in urmatoarea generatie fara ai fi aplicati operatorii de crossover sau de mutatie.
- `Starting Energy`: cantitatea de energie cu care incepe fiecare membru din populatie.
- `Energy For Food`: cata energie este obtinuta la coliziunea creaturii cu o sursa de mancare.
- `Starting Population`: cate creaturi sunt in populatia initiala a fiecarei generatii.

Capitolul 6

Rezultate

Concluzii

Bibliografie

[Rec73] Ingo Rechenberg, *Evolutionsstrategie; Optimierung technischer Systeme nach Prinzipien der biologischen Evolution. Mit einem Nachwort von Manfred Eigen*, Frommann-Holzboog, 1973.

[DM92] D. Dasgupta și D. McGregor, „Designing application-specific neural networks using the structured genetic algorithm", în *Proceedings of the International Conference on Combinations of Genetic Algorithms and Neural Networks*, ed. de D. Whitley și J. D. Schaffer, IEE Press, 1992, pp. 87–96.

[Hol92] John H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control and Artificial Intelligence*, Cambridge, MA, USA: MIT Press, 1992, ISBN: 0262082136.

[Gru93] Frederic Gruau, „Genetic Synthesis of Modular Neural Networks", în *Proceedings of the 5th International Conference on Genetic Algorithms, ICGA-93*, ed. de Stephanie Forrest, University of Illinois at Urbana-Champaign: Morgan Kaufmann, Iul. 1993, pp. 318–325, ISBN: 1-55860-299-2, URL: http://dl.acm.org/citation.cfm?id=645513.657409.

[MM95] David Moriarty și Risto Miikkulainen, „Efficient Reinforcement Learning Through Symbiotic Evolution", în *Machine Learning* 22 (Ian. 1995), DOI: 10.1023/A:1018004120707.

[GM99] Faustino Gomez și Risto Miikkulainen, „Solving Non-Markovian Control Tasks with Neuro-Evolution.", în Ian. 1999, pp. 1356–1361.

[SM02] Kenneth O. Stanley și Risto Miikkulainen, „Evolving Neural Networks through Augmenting Topologies", în *Evolutionary Computation* 10.2 (2002), pp. 99–127, DOI: 10.1162/106365602320169811.

[Uni03] Department of Computer Sciences University of Texas at Austin, *Neuro Evolving Robotic Operatives*, June 16, 2023, 2003, URL: https://nn.cs.utexas.edu/nero/.

[YLH04] G.N. Yannakakis, J. Levine și J. Hallam, „An evolutionary approach for interactive computer games", în *Proceedings of the 2004 Congress on Evolutionary Computation (IEEE Cat. No.04TH8753)*, vol. 1, 2004, 986–993 Vol.1, DOI: 10.1109/CEC.2004.1330969.

[SBM05] K.O. Stanley, B.D. Bryant și R. Miikkulainen, „Real-time neuroevolution in the NERO video game", în *IEEE Transactions on Evolutionary Computation* 9.6 (2005), pp. 653–668, DOI: 10.1109/TEVC.2005.856210.

[Hor+06] Greg Hornby et al., „Automated Antenna Design with Evolutionary Algorithms", în *Collection of Technical Papers - Space 2006 Conference* 1 (Sept. 2006), DOI: 10.2514/6.2006-7242.

[Oro13] Daniel Oro, „Grand challenges in population dynamics", în *Frontiers in Ecology and Evolution* 1 (2013), ISSN: 2296-701X, DOI: 10.3389/fevo.2013.00002, URL: https://www.frontiersin.org/articles/10.3389/fevo.2013.00002.

[Lag15] Sebastian Lague, *Field of view visualisation (E01)*, Youtube, 2015, URL: https://www.youtube.com/watch?v=rQG9aUWarwE&list=PLFt_AvWsXl0dohbtVgHUY7xZv7.

[Sta15] Kenneth O. Stanley, *The NeuroEvolution of Augmenting Topologies (NEAT) Users Page*, University of Central Florida - College of Engineering și Computer Science, 2015, URL: http://www.cs.ucf.edu/~kstanley/neat.html.

[Gel16] Sanchez M. Gelles T., „Comparing Adaptivity of Ants using NEAT and rt-NEAT", în (2016).

[DeL20] Lyon S. DeLong JP, „Temperature alters the shape of predator-prey cycles through effects on underlying mechanisms.", în *PeerJ* (2020), DOI: 10.7717/peerj.9377.

[Zol21] Zolphite, *Simulated Fish Ecosystem (Neural Networks from Scratch + NEAT Genetic Algorithm) Code Along in Unity*, Youtube, 2021, URL: https://www.youtube.com/watch?v=0O2AUKQPAFc&list=PLC8R0n_dfXXr0fc4NdsInilobpp=iAQB.